Module 8: Portfolio Milestone

CSC450-1 Programming III, Colorado State University – Global Campus

Professor H. Pensado

Portfolio Project: Part 2

Hugo Martinez

April 3, 2026

Introduction

Concurrency allows a program to perform multiple operations at the same time, often by dividing work across separate threads. While this concept is commonly used in more complex systems, this application provides a straightforward example of how threads can be created, managed, and controlled.

In this project, the original C++ implementation is extended by developing an equivalent Java application that follows the same logical structure. In both versions, two threads are used to simulate a counting process. The first thread counts upward from 0 to 20, while the second thread counts downward from 20 to 0. Although both tasks are handled using threads, they are executed in a controlled sequence rather than simultaneously. This is achieved through thread synchronization, ensuring that the second thread only begins after the first thread has completed. This approach highlights not only how concurrency works, but also how it can be managed to produce predictable and organized results.

In addition to implementing the Java version of the application, this project also examines key considerations related to concurrency across both programming languages. These include performance trade-offs, as well as potential security concerns related to string handling and data type usage. By comparing the C++ and Java implementations, this project provides a deeper understanding of how concurrency behaves in different programming environments and the implications of those differences.

Pseudocode

The following pseudocode outlines the overall logic and flow of the application used in both the C++ and Java implementations. In particular, it highlights how the program separates the counting tasks into two distinct functions and uses thread control to ensure they execute in a specific sequence. Although both operations are handled using threads, the design intentionally prevents them from running at the same time by requiring the first thread to complete before the second begins.

This shared structure serves as the foundation for both versions of the program, allowing for a direct comparison between how concurrency is implemented in C++ and Java while maintaining consistent behavior and output. This consistency ensures that any differences observed between the two implementations are due to the programming languages themselves rather than changes in program design.

```
BEGIN PROGRAM

IMPORT necessary libraries for threading and timing

DEFINE FUNCTION countUp
```

```
    FOR counter FROM 0 TO 20
        DISPLAY "Counting Up: " + counter
        PAUSE for a short duration (to simulate work and improve readability)
    END FOR
END FUNCTION

DEFINE FUNCTION countDown
    FOR counter FROM 20 DOWN TO 0
        DISPLAY "Counting Down: " + counter
        PAUSE for a short duration
    END FOR
END FUNCTION

MAIN FUNCTION

    DISPLAY "Starting Thread 1: Count Up"

    CREATE Thread1 TO execute countUp function

    START Thread1

    WAIT for Thread1 to complete execution

    DISPLAY "Thread 1 complete. Starting Thread 2: Count Down"

    CREATE Thread2 TO execute countDown function

    START Thread2

    WAIT for Thread2 to complete execution

    DISPLAY "Program complete"

END MAIN
```

```
END PROGRAM
```

C++ Implementation (Part I)

The C++ implementation serves as the baseline for this project, establishing the core logic and structure that is replicated in the Java version. As outlined in the pseudocode, the program uses two threads to perform the counting tasks, with each thread assigned a specific function. The first thread is responsible for counting upward from 0 to 20, while the second thread counts downward from 20 to 0.

A key aspect of this implementation is the use of the join() function to control the execution of the threads. This ensures that the first thread completes its execution before the second thread begins, preventing any overlap and maintaining a clear and predictable output. This is a critical part of the design, as it demonstrates how thread synchronization can be used to enforce sequential behavior within a concurrent program.

Additionally, each counting task is separated into its own function, improving readability and aligning with how threads are typically assigned distinct units of work. This structure provides a clean and consistent foundation that can be directly translated into other programming languages, such as Java.

```
#include <iostream>
#include <thread>
#include <chrono>

using namespace std;

// Function for Thread 1: Count up from 0 to 20
void countUp() {
    for (int i = 0; i <= 20; i++) {
        cout << "Counting Up: " << i << endl;
        this_thread::sleep_for(chrono::milliseconds(200)); // Pause for readability
    }
}

// Function for Thread 2: Count down from 20 to 0
void countDown() {
```

```
    for (int i = 20; i >= 0; i--) {
        cout << "Counting Down: " << i << endl;
        this_thread::sleep_for(chrono::milliseconds(200)); // Pause for readability
    }
}

int main() {
    cout << "Starting Thread 1: Count Up" << endl;

    // Create and start Thread 1
    thread thread1(countUp);
    thread1.join(); // Wait for Thread 1 to finish

    cout << "Thread 1 complete. Starting Thread 2: Count Down" << endl;

    // Create and start Thread 2
    thread thread2(countDown);
    thread2.join(); // Wait for Thread 2 to finish

    cout << "Program complete" << endl;

    return 0;
}
```

Java Implementation (Part II)

The Java implementation follows the same logical structure established in the C++ version, using the shared pseudocode as a foundation. By maintaining the same program design, both implementations produce identical behavior and output, allowing for a direct comparison between the two programming languages.

Similar to the C++ implementation, the program utilizes two threads to perform the counting tasks. The first thread counts upward from 0 to 20, while the second thread counts downward from 20 to 0. Thread execution is controlled using the join() method, which ensures that the first thread completes before the second thread begins. This enforces a sequential flow and prevents any overlap between the two operations.

One notable difference in Java is the way threads are created and managed. In this implementation, threads are created using the Thread class and executed through lambda expressions, which allows the program to pass the counting functions as runnable tasks. Additionally, Java requires exception handling when using methods such as Thread.sleep(), which introduces a level of built-in error management not present in the same way in C++. This implementation demonstrates how the same concurrency concepts can be applied across different programming environments while highlighting differences in syntax, structure, and built-in safeguards.

```
public class CounterApp {


    public static void countUp() {

        for (int i = 0; i <= 20; i++) {

            System.out.println("Counting Up: " + i);

            try {

                Thread.sleep(200);

            } catch (InterruptedException e) {

                System.out.println("Thread interrupted");

            }

        }

    }


    public static void countDown() {

        for (int i = 20; i >= 0; i--) {

            System.out.println("Counting Down: " + i);

            try {

                Thread.sleep(200);

            } catch (InterruptedException e) {

                System.out.println("Thread interrupted");

            }

        }

    }


    public static void main(String[] args) {


        System.out.println("Starting Thread 1: Count Up");
```

```
        Thread thread1 = new Thread(() -> countUp());
        thread1.start();

        try {
            thread1.join();
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }

        System.out.println("Thread 1 complete. Starting Thread 2: Count Down");

        Thread thread2 = new Thread(() -> countDown());
        thread2.start();

        try {
            thread2.join();
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }

        System.out.println("Program complete");
    }
}
```

Program Execution (C++ & Java)

The following outputs demonstrate the execution of both the C++ and Java implementations of the counter application. In each case, the program performs the same sequence of operations, with the first thread counting upward from 0 to 20, followed by the second thread counting downward from 20 to 0.

As shown in the outputs below, both implementations produce identical results, confirming that the same program logic was successfully applied across both programming languages. In each case, thread synchronization is achieved using the join() function in C++ and the join() method in Java, ensuring that the second thread does not begin execution until the first thread has completed.

This consistency in behavior reinforces that any differences between the two implementations are not due to changes in program design, but rather differences in how each language handles concurrency.

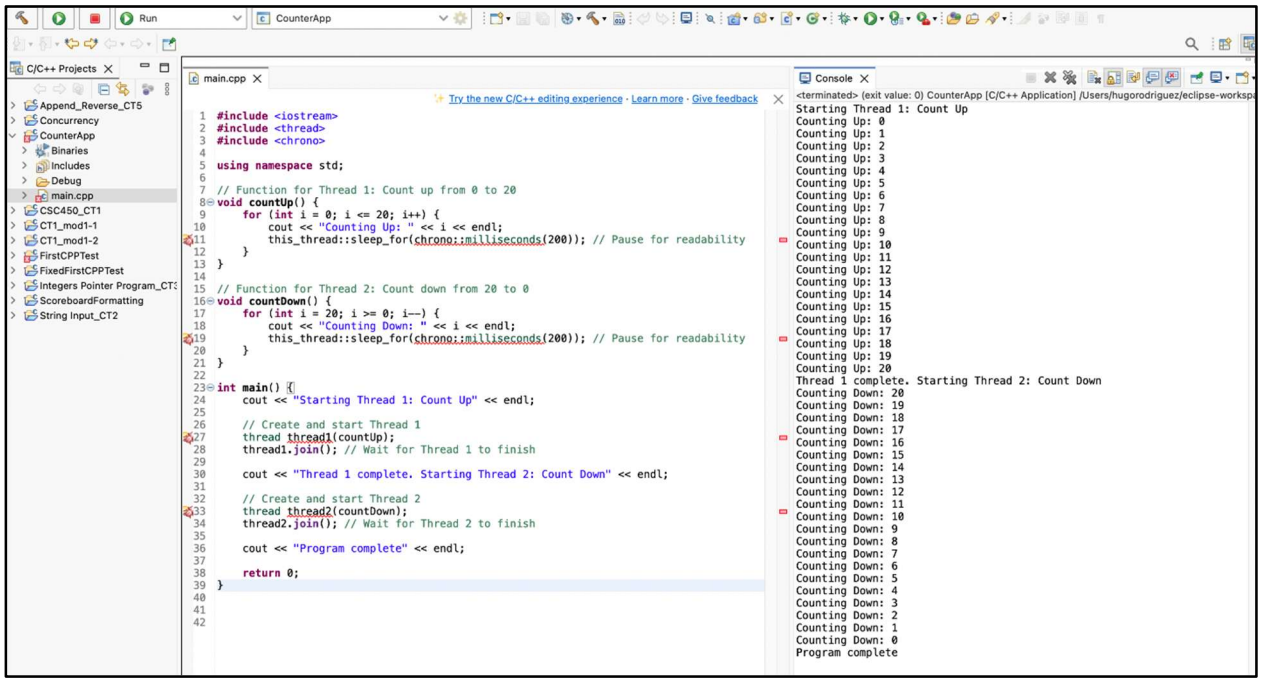


Figure 1. Program execution in C++ performing Count Up and Count Down instruction.

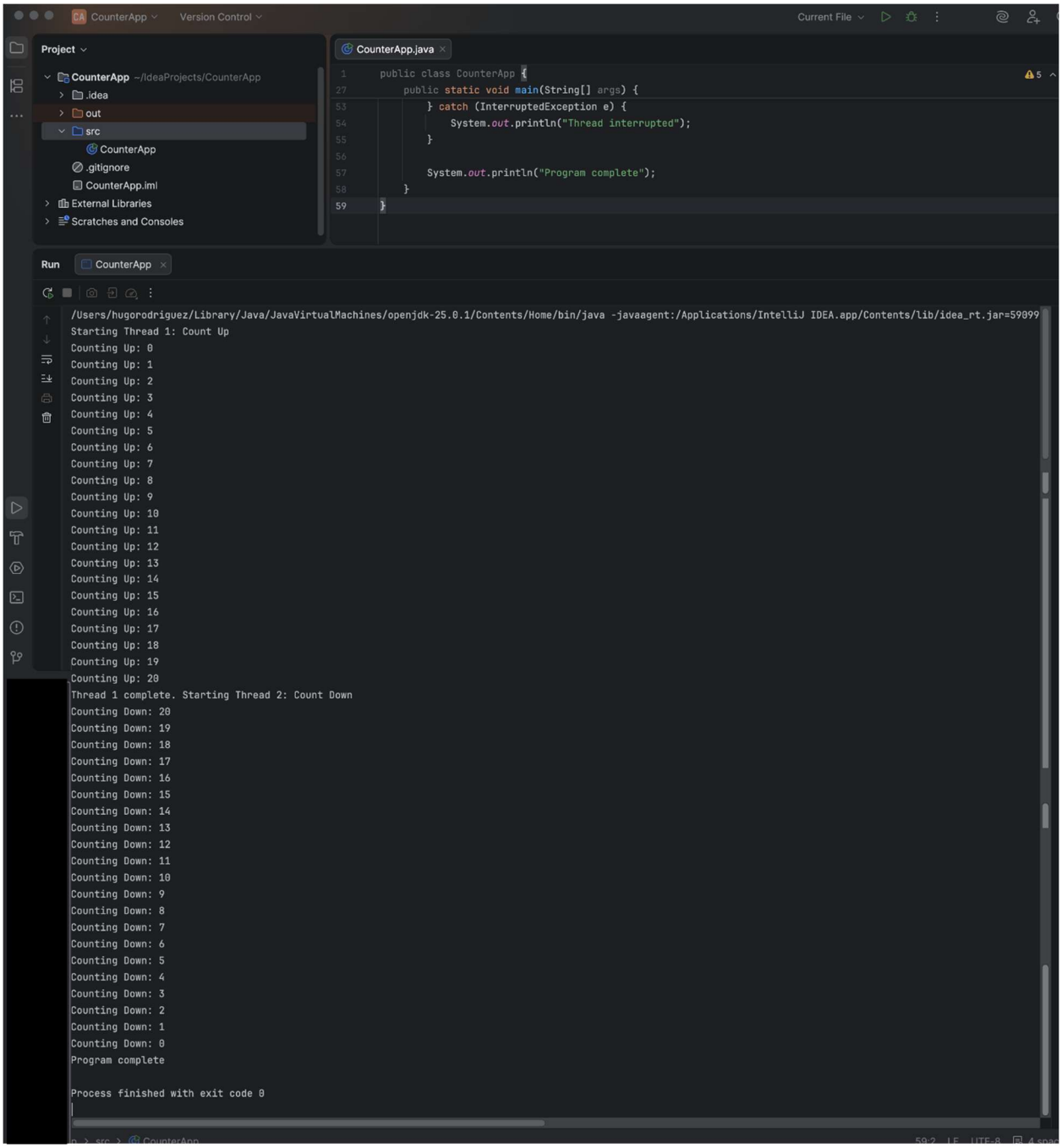


Figure 2. Program execution in Java demonstrating identical sequential thread behavior.

Program Analysis

While the implementations of the program are relatively simple, they highlight several important considerations that come with using concurrency in real-world applications. Beyond just making the program function correctly, it is important to understand how threading can

impact performance, as well as how certain design choices, such as string handling and data type selection can introduce potential vulnerabilities.

Since both the C++ and Java versions follow the same program structure, they provide a useful basis for examining how these considerations apply across different programming environments. The following sections explore these areas and how they relate to both implementations.

1. **Performance Issues with Concurrency**

Concurrency is often associated with performance improvements, as it allows multiple tasks to execute simultaneously. However, the impact on performance depends heavily on how threads are implemented and managed, as well as the underlying characteristics of the programming language.

In both the C++ and Java implementations of this program, concurrency does not provide a performance benefit because the threads are intentionally executed in sequence using the join() function and method. This design ensures predictable and organized output, but it also means that the program does not take advantage of true parallel execution. Instead of improving performance, the use of threads in this case introduces additional overhead related to thread creation and management.

While both languages experience this overhead, the way they handle it differs. C++ operates closer to the system level, allowing for more direct control over thread management and potentially more efficient execution. This can result in better performance in scenarios where fine-tuned optimization is required. In contrast, Java runs within the Java Virtual Machine (JVM), which introduces an additional layer of abstraction. Although this can lead to slightly higher overhead, it also provides benefits such as portability, built-in thread management, and easier development.

This comparison highlights that concurrency is not inherently a performance enhancement. Its effectiveness depends on how it is applied and the environment in which it runs. In this program, concurrency is used primarily to demonstrate thread control and sequencing rather than to improve execution speed.

2. **Vulnerabilities with Strings**

String handling is an important consideration when evaluating program security, as improper handling can introduce vulnerabilities in both C++ and Java, although the level of risk differs between the two languages.

In C++, string handling can be more vulnerable if not managed carefully, particularly when using low-level character arrays. Issues such as buffer overflows can occur when input exceeds

allocated memory space, potentially leading to unstable behavior or security risks. This is largely due to the level of control C++ provides over memory management, which is powerful, but requires extra attention to detail from the developer.

In contrast, Java reduces many of these risks through its built-in String class, which manages memory automatically and prevents direct manipulation of memory locations. This significantly lowers the likelihood of issues such as buffer overflow. However, this does not eliminate all potential risks. For example, improper handling of user input, lack of validation, or inefficient string operations can still lead to vulnerabilities such as injection attacks or performance issues.

In this program, both implementations use simple output statements without user input or dynamic string manipulation, which minimizes risk. However, the comparison highlights that while C++ offers greater flexibility, Java provides stronger built-in safeguards that help reduce the likelihood of common string-related vulnerabilities.

3. **Security of Data Types**

The selection and use of data types play an important role in both program reliability and security. While this program uses simple integer values for counting operations, the way data types are handled can vary significantly between C++ and Java, influencing both performance and risk.

In C++, developers have more direct control over data types and memory usage. This allows for greater flexibility and efficiency, but it also introduces potential risks if not managed carefully. For example, improper use of data types can lead to issues such as integer overflow, unintended data manipulation, or memory-related errors. Because C++ does not enforce the same level of runtime checks, it places greater responsibility on the developer to ensure that data is handled safely.

In contrast, Java provides additional safeguards through its type system and runtime environment. Features such as automatic memory management, built-in type checking, and exception handling help reduce the likelihood of errors related to improper data usage. While Java still requires developers to choose appropriate data types, the language itself helps prevent many common issues that might occur in lower-level languages.

In this program, the use of integer variables within a controlled range minimizes risk in both implementations. However, the comparison highlights that while C++ offers greater control and potential performance advantages, Java provides stronger built-in protections that contribute to overall program safety.

Performance Comparison

Both the C++ and Java implementations of the counter application were developed using the same underlying logic and structure, as defined in the shared pseudocode. In each version, two threads are used to perform simple counting tasks, with one thread counting upward from 0

to 20 and the other counting downward from 20 to 0. In both cases, thread execution is controlled to ensure that the second thread begins only after the first has completed.

Because the structure and behavior of both programs are intentionally kept the same, the resulting output is identical across both implementations. This consistency is important, as it allows for a more accurate comparison between the two programming languages. Any differences observed between the C++ and Java versions can be attributed to how each language handles concurrency, rather than differences in program design. By keeping the logic consistent, this comparison focuses on the underlying characteristics of each language, including how they manage threads, handle performance, and address potential security concerns. This provides a clearer understanding of how the same problem can be approached in different programming environments while producing the same result.

While both the C++ and Java implementations use threads to perform the same tasks, the way each language handles concurrency differs in structure and level of abstraction. These differences affect how developers create, manage, and control threads within each environment. In C++, threads are created using the std::thread library, which provides a relatively direct interface for managing concurrent execution. This approach gives developers more control over how threads are handled, but it also requires a deeper understanding of thread behavior and careful management to avoid issues such as improper synchronization or resource conflicts. The flexibility offered by C++ can be beneficial in performance-critical applications, but it comes with increased responsibility.

In contrast, Java provides a more structured approach to concurrency through its Thread class and support for runnable tasks. In this implementation, threads are created using lambda expressions, which allows the program to pass methods as executable tasks in a concise way. Java also enforces exception handling, requiring developers to account for potential interruptions during thread execution. This built-in structure helps reduce the likelihood of common errors and makes thread management more accessible, particularly for developers working in larger or more complex systems.

Another key difference is how each language integrates concurrency into its overall runtime environment. In C++, thread management is handled more directly by the operating system, while in Java, the JVM plays a significant role in managing thread execution. This results in a more controlled and consistent environment in Java, though it may introduce some additional overhead compared to C++. In this context, both languages provide effective tools for implementing concurrency, but they differ in how much control they give to the developer versus how much is handled by the runtime environment. C++ offers greater flexibility and control, while Java emphasizes structure, safety, and ease of use.

In addition to performance and execution, security is another important consideration when comparing programming languages, particularly in how they handle memory, strings, and data types. While both C++ and Java can be used to build secure applications, the level of risk often depends on how much control is given to the developer versus how much is managed by the language itself.

One of the most significant differences between the two languages is memory management. In C++, developers have direct control over memory allocation and deallocation, which allows for greater flexibility and performance optimization. However, this also introduces potential risks such as memory leaks, dangling pointers, and buffer overflows if not handled carefully. In contrast, Java uses automatic memory management through garbage collection, which reduces the likelihood of these types of errors by handling memory allocation and cleanup internally.

String handling is another area where differences in security become apparent. As discussed earlier, C++ can be more vulnerable when using low-level character arrays, where improper handling can lead to buffer overflow vulnerabilities. Java, on the other hand, uses a built-in String class that manages memory automatically and prevents direct manipulation of memory locations. While this reduces the risk of certain vulnerabilities, Java applications are still susceptible to issues such as improper input validation or inefficient string handling if not implemented carefully.

Data type handling also contributes to overall program security. In C++, the flexibility of data types allows for efficient use of memory and performance tuning, but it can also lead to issues such as overflow or unintended data manipulation if not properly managed. Java provides stronger type enforcement and runtime checks, which help prevent many common errors related to incorrect data usage. This added layer of protection makes Java generally more resilient to certain types of programming mistakes.

This comparison highlights a key trade-off between the two languages. C++ offers greater control and performance potential, but it requires careful attention to detail to maintain security. Java, by contrast, provides built-in safeguards that reduce the likelihood of common vulnerabilities, making it a safer choice in many scenarios, particularly for applications where stability and maintainability are priorities.

Another consideration when looking at trade-offs is that the choice between the two often depends on the specific requirements of the application rather than one language being universally better than the other. Each language offers its own advantages, and understanding these trade-offs is important when deciding which to use. C++ is often preferred in situations where performance and system-level control are critical. Because it operates closer to the hardware and allows direct memory management, it is commonly used in applications such as game development, embedded systems, and high-performance computing. This level of control can lead to more efficient execution, but it also requires a higher level of expertise to manage safely and effectively.

On the other hand, Java is widely used in applications where stability, portability, and ease of development are priorities. The use of the Java Virtual Machine allows Java programs to run consistently across different systems, making it a strong choice for enterprise applications, web services, and large-scale systems. Additionally, Java's built-in memory management and structured approach to concurrency reduce the likelihood of common programming errors, which can improve long-term maintainability.

In the context of this project, both languages successfully achieve the same result using similar logic. However, the comparison highlights that while C++ provides more control and potential for performance optimization, Java offers a more managed environment that simplifies development and enhances safety. The decision between the two ultimately depends on whether the priority is performance and control or reliability and ease of use.

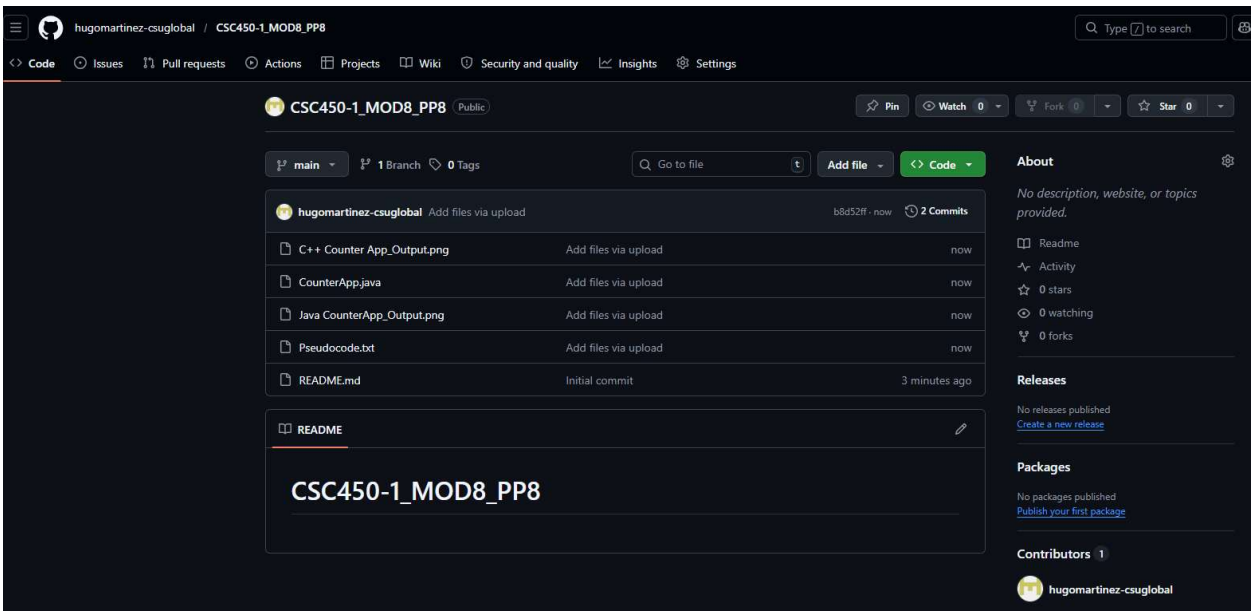
The comparison between the C++ and Java implementations demonstrates that both languages are capable of achieving the same functional outcome when applying the same program logic. By maintaining a consistent structure across both versions, this project highlights how differences in performance, concurrency handling, and security are primarily influenced by the design and characteristics of each programming language rather than the problem itself.

C++ offers greater control over system resources and the potential for higher performance, but it requires careful management to avoid common issues related to memory and data handling. Java, in contrast, provides a more structured and managed environment, reducing the likelihood of these issues through built-in safeguards such as automatic memory management and strong type enforcement.

Overall, the results of this project show that the choice between C++ and Java is not about which language is better, but rather which is more appropriate for a given situation. Both languages provide effective tools for implementing concurrency, and understanding their differences allows developers to make more informed decisions based on the needs of their applications.

GitHub Repository

The submission details can be found at the following GitHub Repository link: https://github.com/hugomartinez-csuglobal/CSC450-1_MOD8_PP8



Conclusion

This project demonstrated how the same concurrency concepts can be implemented across different programming languages while producing consistent and predictable results. By developing both C++ and Java versions of the application using a shared design, it became clear that differences in behavior are not driven by the logic of the program, but by how each language manages execution, memory, and system resources.

Through this comparison, it is evident that understanding the strengths and limitations of each language is just as important as writing functional code. While both implementations successfully achieved the intended outcome, the process highlights the importance of considering factors such as performance, control, and safety when selecting a programming approach. Working through this project reinforced the idea that effective software development is not only about solving a problem, but also about choosing the right tools and understanding how those tools influence the final output.